

强化学习原理：动态规划方法

郭宪

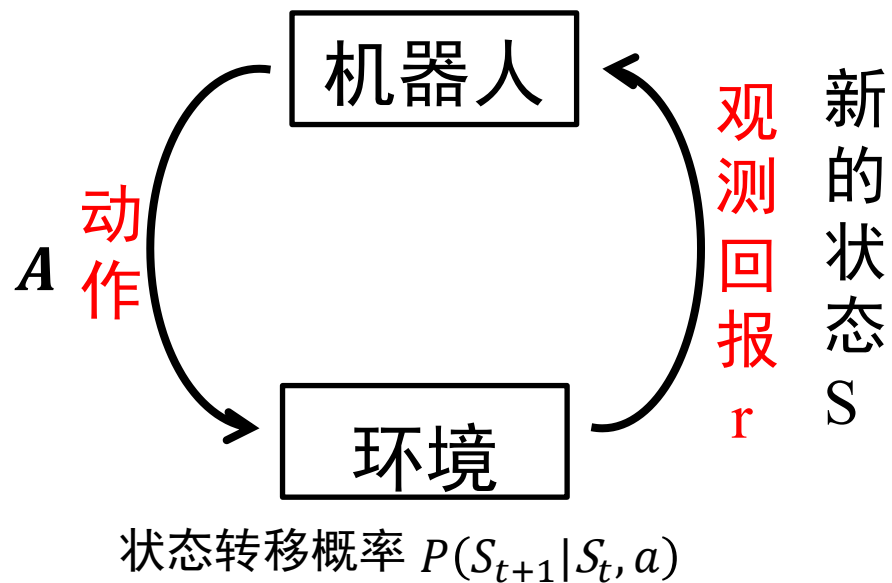
人工智能学院

College of Artificial Intelligence



南开大学
Nankai University

强化学习的形式化表示



马尔科夫决策问题 (MDP): 四元组 (S, A, P, r)

策略: $\pi: S \rightarrow u$ 。常采用随机策略: $\pi(u|s)$

累积回报: $R(\tau) = r_T(x_T) + \sum_{t=0}^{T-1} r_t(x_t, u_t)$

折扣回报: $R = \sum_{t=0}^{\infty} \gamma^t r_t$ 值函数

强化学习目标: $\max_{\pi} \int R(\tau) p_{\pi}(\tau) d\tau$

值函数最优时采取的策略是最优的, 反过来, 策略最优时, 值函数最优。



动态规划的本质

离散贝尔曼方程：

$$\begin{aligned} J^*[x(j), j] &= \min_{u(j) \in U} \min_{\{u(j+1), \dots, u(N-1)\} \in U} \{L[x(j), u(j), j] + \sum_{k=j+1}^{N-1} L[x[k], u[k], k]\} \\ &= \min_{\substack{u(j) \in U \\ x(j+1) \in X}} \{L(x(j), u(j), j) + J^*[x(j+1), j+1]\} \\ &= \min_{\substack{u(j) \in U \\ x(j+1) \in X}} \{L(x(j), u(j), j) + J^*[f[x(j), u(j), j], j+1]\} \end{aligned}$$

求出值函数后，通过**贪婪策略**重构出最优策略

动态规划的本质是：将多阶段决策问题通过贝尔曼方程转化为多个单阶段的决策问题

动态规划：多阶段决策

最短路径问题：从 q 到 r 的最短路径

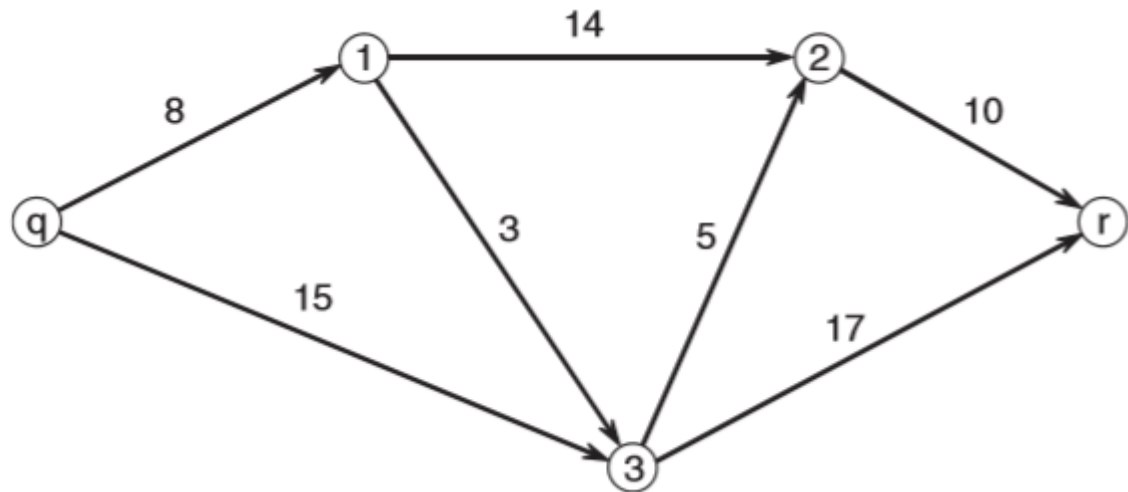


Table 1.1 Path cost from each node to node r after each node has been visited

Iteration	Cost from Node				
	q	1	2	3	r
	100	100	100	100	0
1	100	100	10	15	0
2	30	18	10	15	0
3	26	18	10	15	0
4	26	18	10	15	0

$$J^*[x(j), j] = \min_{\substack{u(j) \in U \\ x(j+1) \in X}} \{L(x(j), u(j), j) + J^*[x(j+1), j+1]\}$$

值函数的更新公式为：

$$v_i \leftarrow \min \left\{ v_i, \min_{j \in I^+} \{c_{ij} + v_j\} \right\}$$

Warren B. Powell,
Approximate Dynamic Programming: Solving the
Curses of Dimensionality



最优控制中的动态规划

动态规划的本质是：将多阶段决策问题通过贝尔曼方程转化为多个单阶段的决策问题

连续贝尔曼方程：

$$J^*[x(t), t] = \min_{u[t, t+\Delta t]} \left\{ \min_{u[t+\Delta t, t_f]} \left[\int_t^{t+\Delta t} L(x(\tau), u(\tau), \tau) d\tau + \int_{t+\Delta t}^{t_f} L(x(\tau), u(\tau), \tau) d\tau + \phi[x(t_f), t_f] \right] \right\}$$
$$= \min_{\substack{u(\tau) \in U \\ t \leq \tau \leq t+dt}} \left\{ \int_t^{t+dt} L[x(\tau), u(\tau), \tau] d\tau + J^*[x(t) + dx(t), t + dt] \right\} \quad (1)$$

将 $J^*[x(t) + dx(t), t + dt]$ 进行泰勒展开有：

$$J^*[x(t) + dx(t), t + dt] = J^*[x(t), t] + \frac{\partial J^*[x(t), t]}{\partial x^T(t)} dx(t) + \frac{\partial J^*[x(t), t]}{\partial t} dt + \varepsilon[dx(t), dt] \quad (2)$$

将 (2) 带入 (1)，并令 $dt \rightarrow 0$ **Hamilton-Jacobi-Bellman方程**

胡寿松等，最优控制理论与系统，
科学出版社，2005

$$-\frac{\partial J^*[x(t), t]}{\partial t} = \min_{u(t) \in U} \{ L[x(t), u(t), t] + \frac{\partial J^*[x(t), t]}{\partial x^T(t)} f[x(t), u(t), t] \} \xrightarrow{\text{重构}} \min_{u(t) \in U} \{ L[x(t), u(t), t] + \frac{\partial J^*[x(t), t]}{\partial x^T(t)} f[x(t), u(t), t] \}$$



从动态规划到强化学习

所有强化学习算法都可以看成是动态规划算法，只是用更少的计算，并且没有假设完美模型

离散贝尔曼方程：

$$J^*[x(j), j] = \min_{u(j) \in U} \min_{\{u(j+1), \dots, u(N-1)\} \in U} \{L[x(j), u(j), j] + \sum_{k=j+1}^{N-1} L[x[k], u[k], k]\}$$

$$= \min_{\substack{u(j) \in U \\ x(j+1) \in X}} \{L(x(j), u(j), j) + J^*[x(j+1), j+1]\}$$

$$= \min_{\substack{u(j) \in U \\ x(j+1) \in X}} \{L(x(j), u(j), j) + J^*[f[x(j), u(j), j], j+1]\}$$

策略改进

策略评估

连续贝尔曼方程：

策略评估

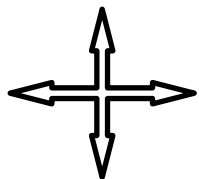
策略改进

重构

$$-\frac{\partial J^*[x(t), t]}{\partial t} = \min_{u(t) \in U} \{L[x(t), u(t), t] + \frac{\partial J^*[x(t), t]}{\partial x^T(t)} f[x(t), u(t), t]\} \xrightarrow{\text{重构}} \min_{u(t) \in U} \{L[x(t), u(t), t] + \frac{\partial J^*[x(t), t]}{\partial x^T(t)} f[x(t), u(t), t]\}$$

策略评估：值函数

MDP



动作

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

状态空间: $S=\{1,2..14\}$

动作空间: {东, 南, 西, 北}

回报函数: -1, 直到终止状态

均匀随机策略:

$$\pi(\text{东}|\cdot)=0.25, \quad \pi(\text{南}|\cdot)=0.25, \quad \pi(\text{西}|\cdot)=0.25, \\ \pi(\text{北}|\cdot)=0.25$$

当智能体采用策略 π 时, 累积回报服从一个概率分布, 累积回报在状态 s 处的期望值定义为值函数:

$$v_{\pi}(s) = E_{\pi}[G_t | S_t = s] = E_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right]$$

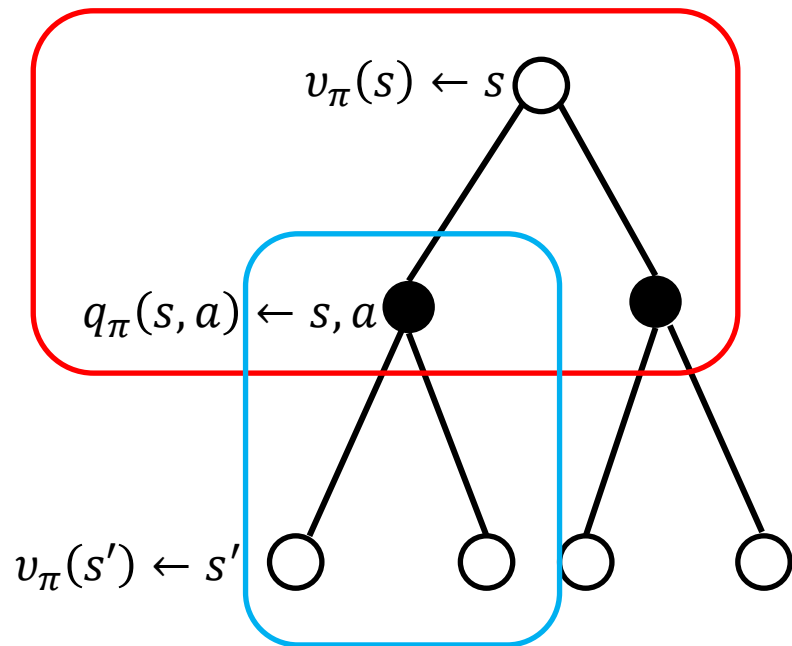
状态-行为值函数:

$$q_{\pi}(s, a) = E_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a \right]$$

0.0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0.0

策略评估(policy evaluation)

给定策略 π 构造值函数:



$$v_{\pi}(s) = \sum_{a \in A} \pi(a|s) q_{\pi}(s, a)$$

$$q_{\pi}(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi}(s')$$



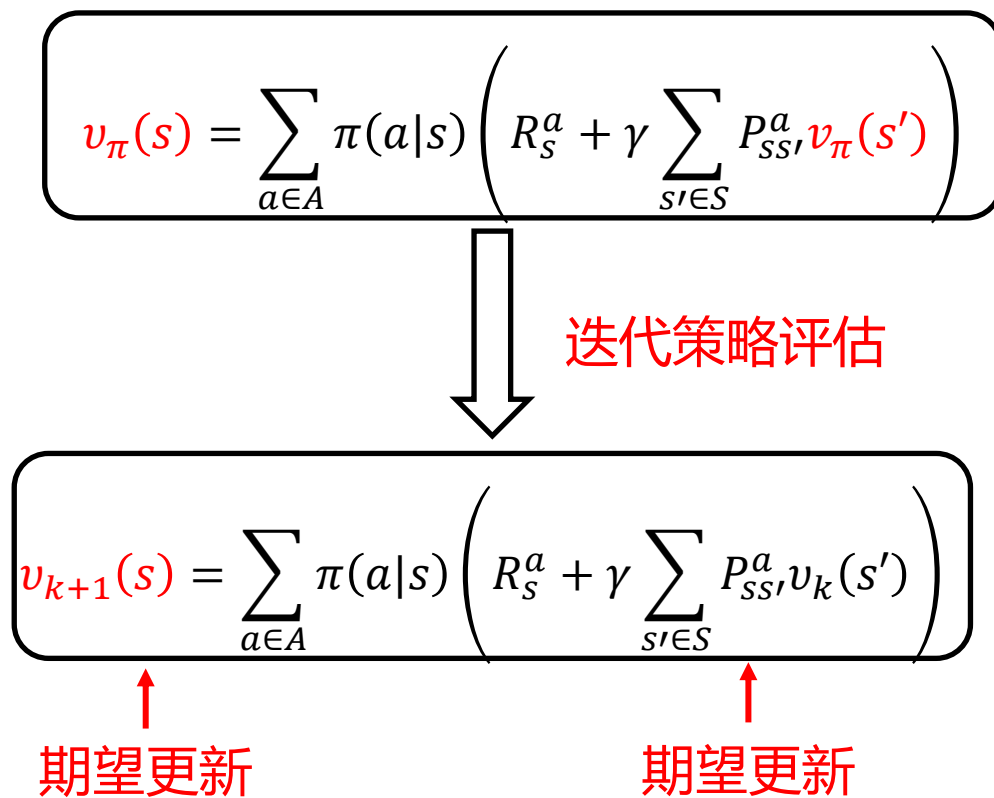
$$v_{\pi}(s) = \sum_{a \in A} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi}(s') \right)$$

模型已知，方程组中只有值函数是未知数，方程组是线性方程组。未知数的数目等于状态的数目。



采用数值迭代算法

策略评估(policy evaluation)



解析法:

$$V_\pi(S) = C_\pi(S) + \gamma P^\pi V_\pi(S)$$

其中:

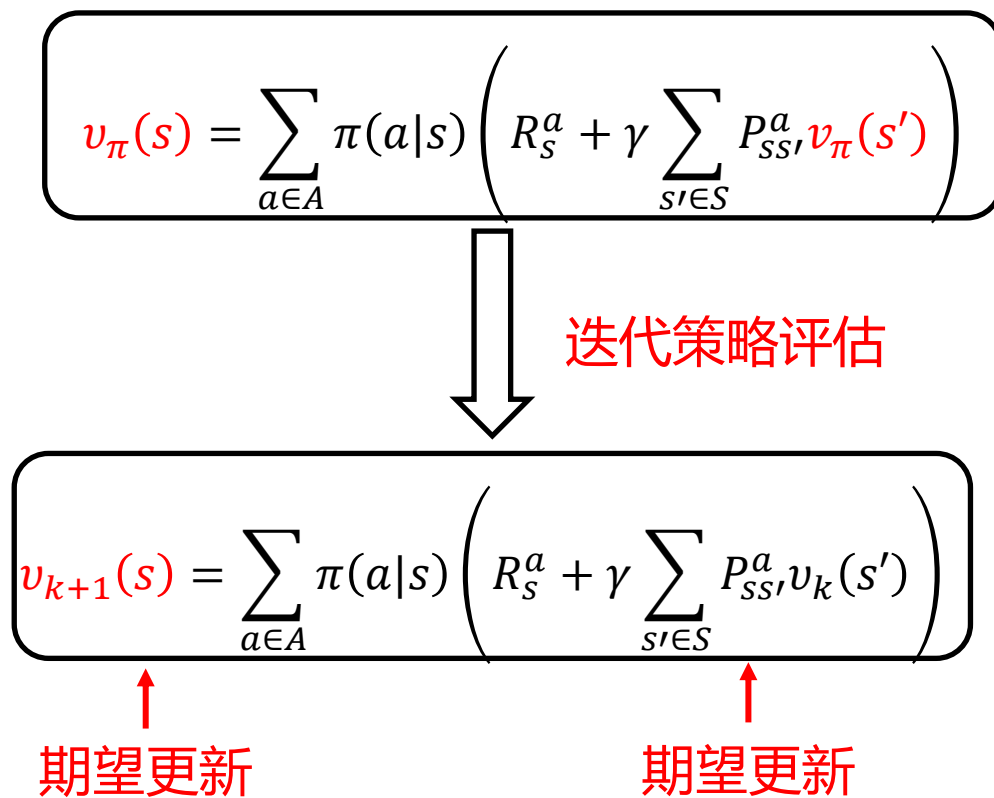
$$C_\pi = \sum_a \pi(a|s) r(s, a)$$

$$P^\pi = \sum_a \pi(a|s) \sum_{s'} p(s'|s, a)$$

值函数为:

$$V_\pi(s) = (I - \gamma P^\pi)^{-1} C_\pi(s)$$

策略评估(policy evaluation)



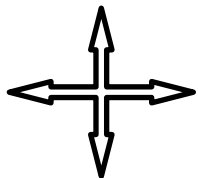
策略评估算法：数值迭代算法

- [1] 输入：需要评估的策略 π 状态转移概率 $P_{ss'}^a$, 回报函数 R_s^a , 折扣因子 γ
- [2] 初始化值函数： $V(s) = 0$
- [3] Repeat $k=0,1,\dots$
- [4] for **every** s do
- [5]
$$v_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_k(s') \right)$$
- [6] end for
- [7] Until $v_{k+1} = v_k$
- [8] 输出： $v(s)$

一次状态扫描

策略评估(policy evaluation)

MDP



动作

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

状态空间: $S=\{1,2..14\}$

动作空间: {东, 南, 西, 北}

回报函数: -1, 直到终止状态

均匀随机策略:

$\pi(\text{东}|\cdot)=0.25$, $\pi(\text{南}|\cdot)=0.25$, $\pi(\text{西}|\cdot)=0.25$,
 $\pi(\text{北}|\cdot)=0.25$

策略评估算法: 解析法

$$V_{\pi}(S)=[v_{\pi}(s=0), v_{\pi}(s=1), v_{\pi}(s=2), \dots, v_{\pi}(s=14), v_{\pi}(s=15)]$$

$$C_{\pi}=\sum_a \pi(a|s)r(s,a)=[0,-1,-1,\dots,-1,0]$$

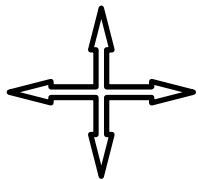
$$P^{\pi}=\sum_a \pi(a|s)\sum_{s'} p(s'|s,a)$$

$$P^{\pi}=\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.25 & 0.25 & 0.25 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.25 & 0.25 & 0.25 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.25 & 0.5 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.25 & 0 & 0 & 0 & 0.25 & 0.25 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.25 & 0 & 0 & 0.25 & 0 & 0.25 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.25 & 0 & 0 & 0.25 & 0 & 0.25 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.25 & 0 & 0 & 0.25 & 0.25 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0.25 & 0.25 & 0 & 0 & 0.25 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.25 & 0 & 0.25 & 0 & 0 & 0.25 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.25 & 0 & 0.25 & 0 & 0 & 0.25 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.25 & 0.25 & 0 & 0 & 0 & 0.25 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0 & 0.5 & 0.25 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.25 & 0.25 & 0.25 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.25 & 0 & 0 & 0.25 & 0.25 & 0.25 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$V_{\pi}[S]=[0,-14,-20,-22,-14,-18,-20,-20,-20,-20,-18,-14,-22,-20,-14,0]$$

策略评估(policy evaluation)

MDP



动作

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

状态空间: $S=\{1,2..14\}$

动作空间: {东, 南, 西, 北}

回报函数: -1, 直到终止状态

均匀随机策略:

$\pi(\text{东}|\cdot)=0.25$, $\pi(\text{南}|\cdot)=0.25$, $\pi(\text{西}|\cdot)=0.25$,
 $\pi(\text{北}|\cdot)=0.25$

策略评估算法

输入: 需要评估的策略 π 状态转移概率 $P_{ss'}^a$, 回报函数 R_s^a , 折扣因子 γ

初始化值函数: $V(s) = 0$

Repeat $k=0,1,\dots$ 一次状态扫描

for every s do

$$v_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_k(s') \right)$$

end for

Until $v_{k+1} = v_k$

输出: $v(s)$

策略评估(policy evaluation)

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

K=0

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

K=1

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

K=2

0.0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0.0

$K = \infty$

策略评估算法

输入：需要评估的策略 π 状态转移概率 $P_{ss'}^a$ 回报函数 R_s^a ，折扣因子 γ

初始化值函数： $V(s) = 0$

Repeat $k=0,1,\dots$

一次状态扫描

for every s do

$$v_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_k(s') \right)$$

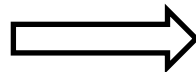
end for

Until $v_{k+1} = v_k$

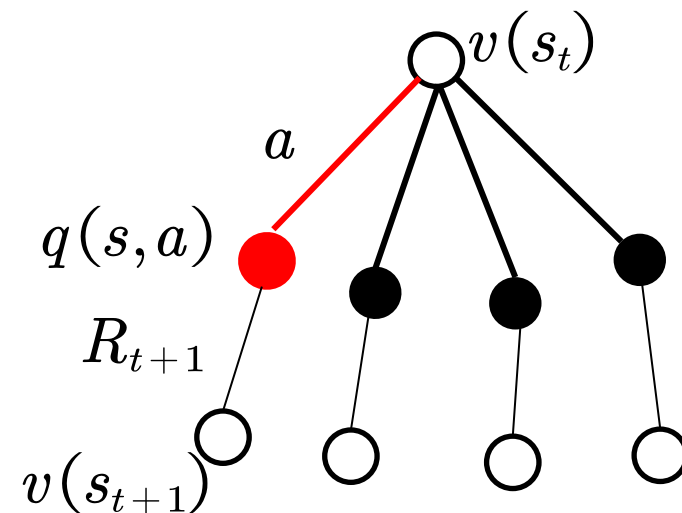
输出： $v(s)$

策略改进(policy improvement)

0.0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0.0

?
 更好的策略 π_1

π_0 均匀策略:



$$q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(s_{t+1}) | s_t = s, a_t = a]$$

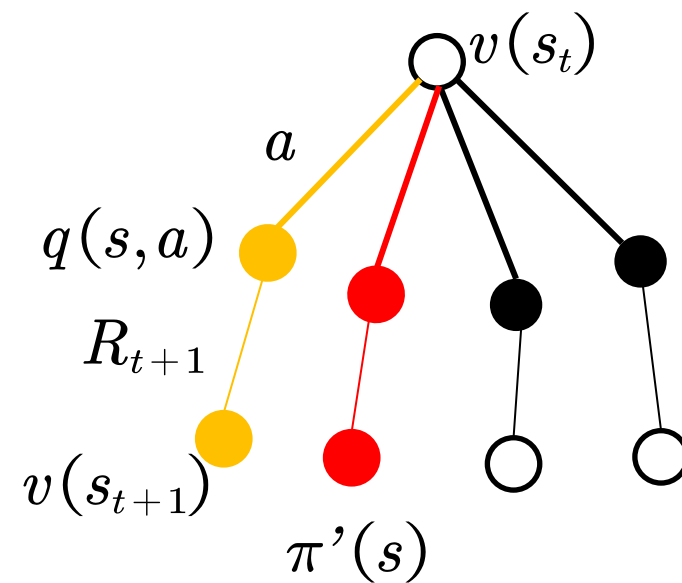
策略改善理论:

如果: $q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s)$

那么: π' 不比 π 差, 甚至比之还要好

策略改进证明(policy improvement)

$$\begin{aligned}
 v_{\pi}(s) &\leq q_{\pi}(s, \pi'(s)) \\
 &= \mathbb{E}[R_{t+1} + \gamma v_{\pi}(s_{t+1}) | s_t = s, a_t = \pi'(s)] \\
 &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_{\pi}(s_{t+1}) | s_t = s] \\
 &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_{\pi}(s_{t+1}, \pi'(s_{t+1})) | s_t = s] \\
 &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma \mathbb{E}_{\pi'}[R_{t+2} + \gamma v_{\pi}(s_{t+2}) | s_{t+1}, a_{t+1} = \pi'(s_{t+1})] | s_t = s] \\
 &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_{\pi}(s_{t+2}) | s_t = s] \\
 &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_{\pi}(s_{t+3}) | s_t = s] \\
 &\vdots \\
 &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots | s_t = s] \\
 &= v_{\pi'}(s)
 \end{aligned}$$



最直观的一个改进策略是什么？

贪婪策略！

策略改进(policy improvement)

计算策略值的目的是为了帮助找到更好的策略，在每个状态采用贪婪策略。

$$\pi_{l+1}(s) \in \underset{a}{\operatorname{argmax}} q^{\pi_l}(s, a)$$

π_0 均匀策略: 

π_1 贪婪策略:

$K=10$

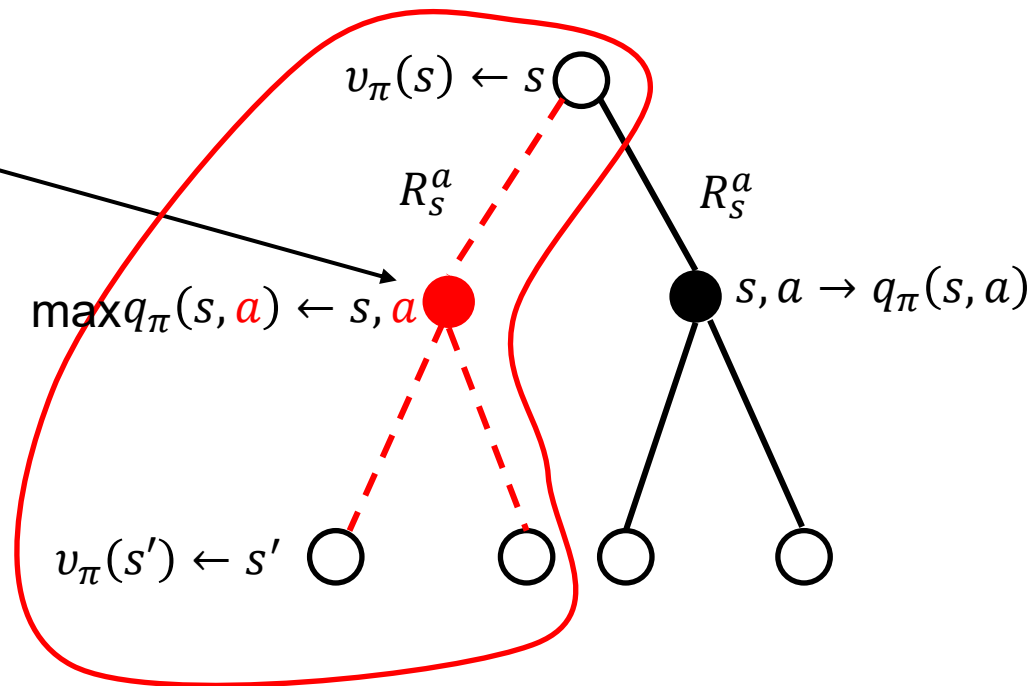
0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

0.0	←	←	↖
↑	↗	↖	↓
↑	↖	↗	↓
↖	→	→	0.0

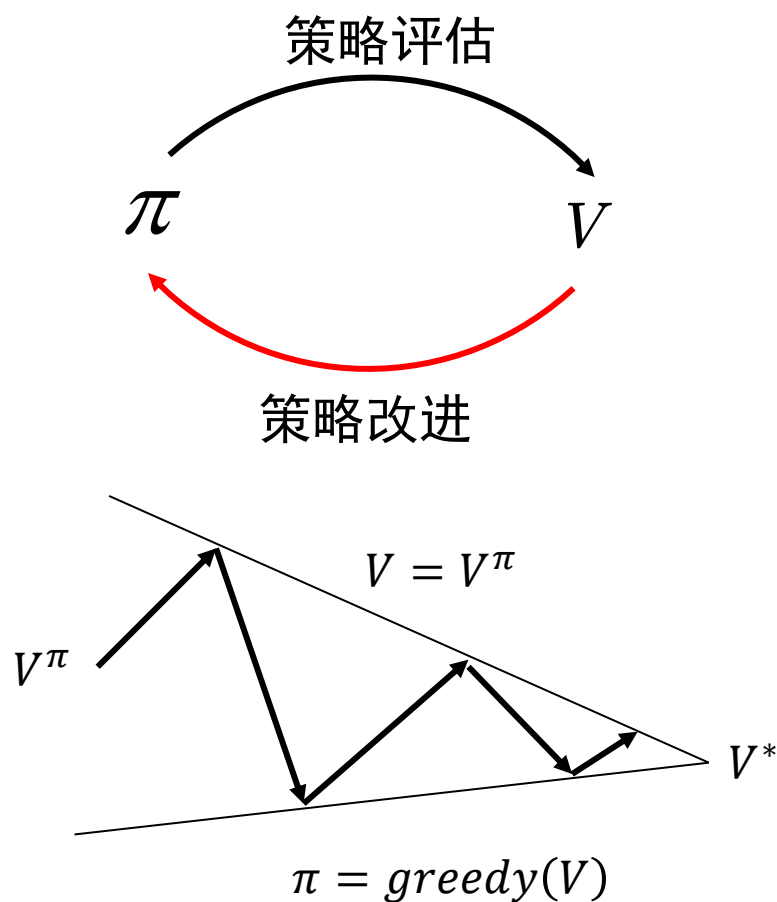
$K = \infty$

0.0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0.0

0.0	←	←	↖
↑	↗	↖	↓
↑	↖	↗	↓
↖	→	→	0.0



策略迭代(policy iteration)



算法1：策略迭代算法

- [1] 输入：状态转移概率 $P_{ss'}^a$, 回报函数 R_s^a , 折扣因子 γ
初始化值函数： $V(s) = 0$ 初始化策略 π_0
- [2] Repeat $l=0,1,\dots$
- [3] find V^{π_l} Policy evaluation
- [4] $\pi_{l+1}(s) \in \arg \max_a q^{\pi_l}(s, a)$ Policy improvement
- [5] Until $\pi_{l+1} = \pi_l$
- [6] 输出： $\pi^* = \pi_l$

值函数迭代

策略改进一定要等到值函数收敛吗？

π_0 均匀策略: 

$K=10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

π_1 贪婪策略:

0.0	←	←	↖
↑	↖	↖	↓
↑	↘	↘	↓
↙	→	→	0.0

$K = \infty$

0.0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0.0

0.0	←	←	↖
↑	↖	↖	↓
↑	↘	↘	↓
↙	→	→	0.0

当 $K=1$ 时便进行策略改进, 得到值函数迭代算法

$$v^*(s) = \max_a R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_*(s')$$

[1] 输入: 状态转移概率 $P_{ss'}^a$, 回报函数 R_s^a , 折扣因子 γ

初始化值函数: $v(s) = 0$ 初始化策略 π_0

[2] Repeat $l=0, 1, \dots$

[3] for **every** s do

[4]
$$v_{l+1}(s) = \max_a R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_l(s')$$

[5] Until $v_{l+1} = v_l$

[6] 输出:
$$\pi(s) = \operatorname{argmax}_a R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_l(s')$$



作业



1. 阅读 Sutton 书第三章， 并做读书笔记